SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS: Total Marks: 30

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
 <input type="text" name="name">
 <input type="email" name="email">
 <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
 <head>
 <title>Company</title>
 </head>
 <body>
 <div>
 <h1>Welcome
 <p>About us
 </div>
 </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
 <input type="text" name="username">
 <input type="password" name="password">
 <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I _____Meghanadh A (192372281)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: meghanadh

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

1. Online Symposium Portal

1. Identify appropriate HTML5 semantic elements missing

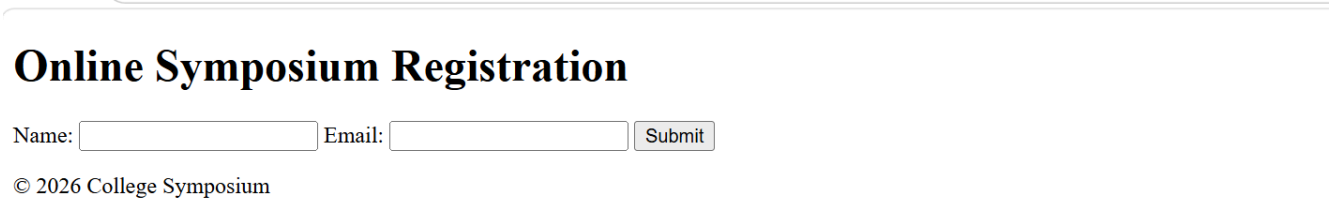
The form should be placed inside semantic HTML5 elements such as:

- <header>
- <main>
- <section>
- <form>
- <label>
- <footer> (optional)

2. Standards-compliant HTML structure

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Symposium Registration</title>
</head>
<body>
<header>
    <h1>Online Symposium Registration</h1>
```

```
</header>
<main>
    <section>
        <form action="/register" method="POST">
            <label for="name">Name:</label>
            <input type="text" id="name" name="name" required>
            <label for="email">Email:</label>
            <input type="email" id="email" name="email" required>
            <button type="submit">Submit</button>
        </form>
    </section>
</main>
<footer>
    <p>&copy; 2026 College Symposium</p>
</footer>
</body>
</html>
```



3. HTTP request generated during form submission

When the user clicks **Submit**, the browser sends an HTTP POST request.

Example:

POST /register HTTP/1.1

Host: www.college.edu

Content-Type: application/x-www-form-urlencoded

name=John&email=john@example.com

POST → Sends data to the server.

/register → Server endpoint.

Body → Contains form data.

4. HTTP response cycle

User fills the form.

Browser sends POST request.

Server receives and validates data.

Server stores registration information.

Server returns an HTTP response.

Example:

HTTP/1.1 200 OK

Content-Type: text/html

Browser displays:

Registration Successful

If validation fails:

HTTP/1.1 400 Bad Request

5. Improvements for usability and accessibility

Use `<label>` for every input.

Add `required` attribute.

Use meaningful placeholder text.

Add client-side validation.

Use HTTPS.

Display success/error messages.

Ensure keyboard navigation.

Provide ARIA attributes where necessary.

2. Cross-Browser Compatibility Issue

1. Improper nesting and missing closing tags

Problems:

- Missing `</h1>`
- Missing `</p>`
- Missing `</div>`
- Improper nesting
- No `<!DOCTYPE html>`
- Missing `<meta charset>` and viewport.

2. Browser interpretation

Different browsers automatically try to fix invalid HTML.

Possible effects:

- Different spacing
- Different font sizes
- Broken layout
- Unexpected rendering
- Inconsistent DOM structure

3. Missing semantic elements

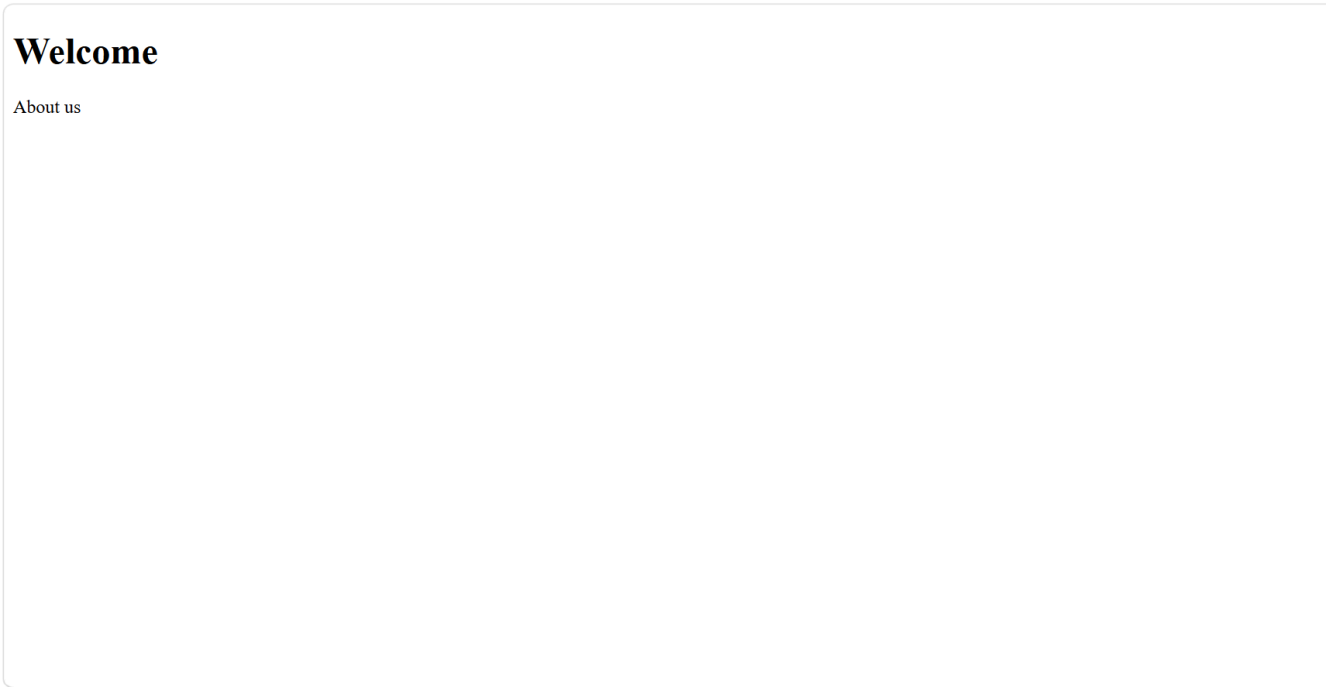
Should include:

- `<header>`
- `<main>`
- `<section>`

4. Correct W3C-compliant HTML

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Company</title>

</head>

<body>

<header>

    <h1>Welcome</h1>
```

```
</header>
<main>
    <section>
        <p>About us</p>
    </section>
</main>
</body>
</html>
```



5. Techniques for cross-browser compatibility

Validate HTML using W3C Validator.

Use HTML5 semantic elements.

Include `<!DOCTYPE html>`.

Test in Chrome, Firefox, Edge, and Safari.

Use CSS Reset or Normalize.css.

Use feature detection instead of browser detection.

Write standards-compliant HTML and CSS.

Use responsive design.

3. Form Submission Failure

1. Issues in the form configuration

Problems:

- Button type is `"button"` instead of `"submit"`.
- GET method sends sensitive data in URL.
- Password should not be transmitted using GET.

2. Why the HTTP request is not triggered

The button is:

<button type="button">

This only performs a normal button action.

It does **not** submit the form.

The correct type is:

<button type="submit">

3. Role of GET method

GET:

- Sends data in the URL.
- Used for retrieving information.
- Data is visible in browser history.
- Not suitable for passwords or confidential information.

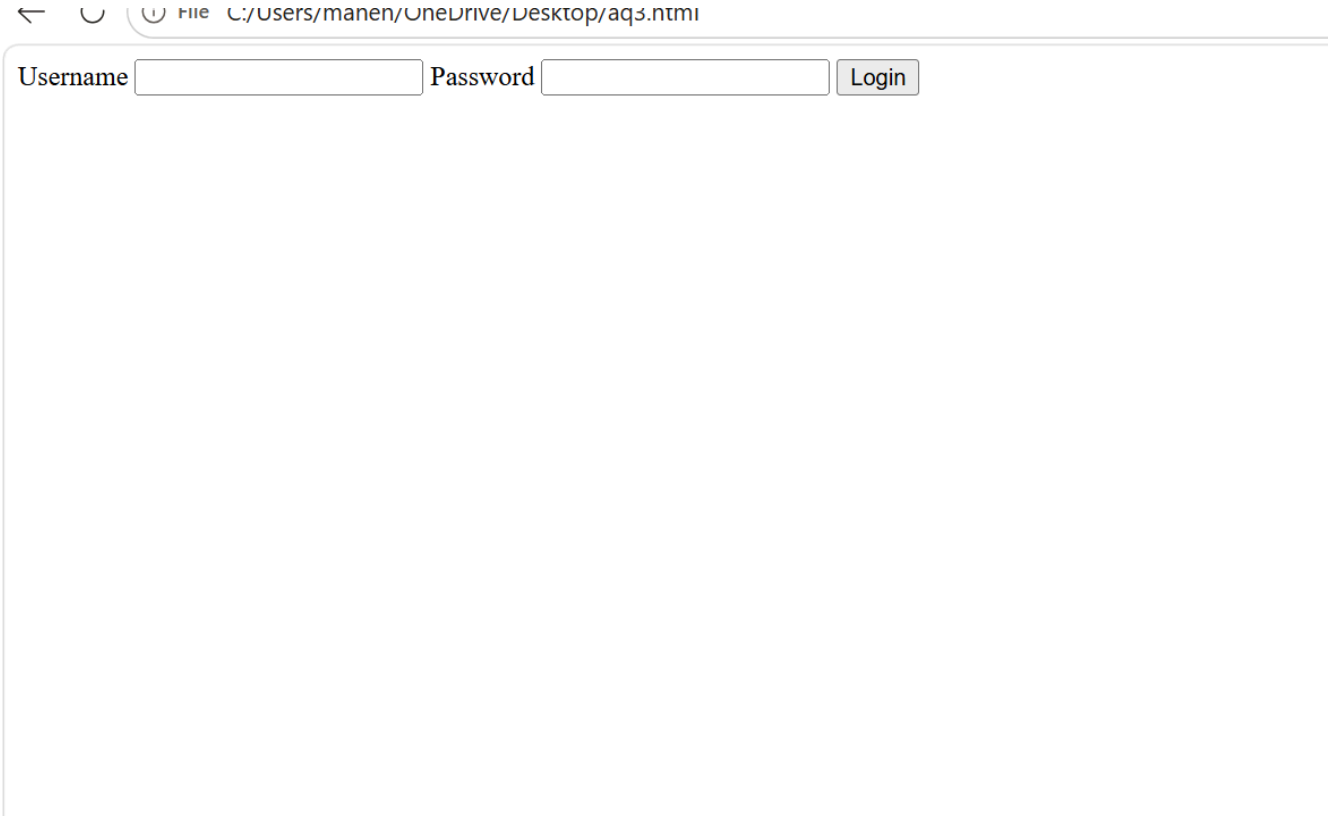
Example:

/submit?username=John&password=12345

4. Correct implementation

<!DOCTYPE html>

```
<html lang="en">
<head>
<meta charset="UTF-8">
<title>Login</title>
</head>
<body>
<form action="/submit" method="POST">
<label for="username">Username</label>
<input type="text" id="username" name="username" required>
<label for="password">Password</label>
<input type="password" id="password" name="password" required>
<button type="submit">Login</button>
</form>
</body>
</html>
```



5. Improvements for secure data transmission

- Use **POST** instead of GET.
- Use **HTTPS** to encrypt communication.
- Validate input on both client and server.
- Hash passwords before storing them.
- Implement CSRF protection.
- Use secure session management.
- Display meaningful error messages without exposing sensitive information.
- Enable server-side authentication and input sanitization.